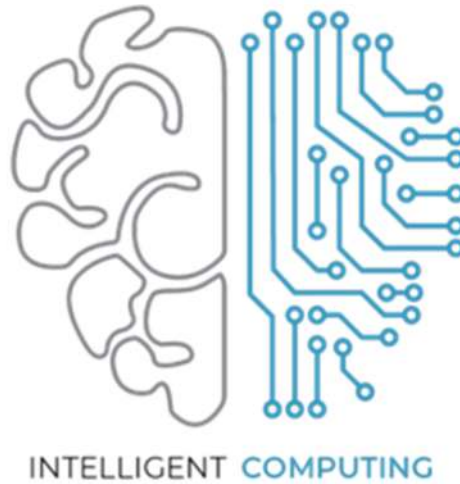


UAS

PENGOLAHAN CITRA



NAMA : Dzaky Rahman Ikhsan

NIM : 202431004

KELAS : A

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC : 25

ASISTEN : 1. Sharira Maharan

2. Muhammad Hanif Nuruddin Jabbar

3. Fhazel Kesra Arivi

4. Muhammad Farhan Fahrezy

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2025/2026

DAFTAR ISI

DAFTAR ISI	2
BAB I	3
PENDAHULUAN.....	3
1.1 Rumusan Masalah	3
1.2 Tujuan Masalah	3
1.3 Manfaat Masalah	3
BAB II	4
LANDASAN TEORI	4
2.1 Pengolahan Citra Digital	4
2.2 Transformasi Geometri Citra	4
2.3 Translasi (Translation)	5
2.4 Rotasi (Rotation)	5
2.5 Scaling	5
2.6 Flipping	6
2.7 OpenCV dan Python	6
BAB III	7
HASIL	7
3.1 Analisis Sistem	7
3.2 Import Library	7
3.3 Membaca Citra	8
3.4 Proses Deteksi Tepi Menggunakan Metode Canny	9
3.5 Proses Pencarian Kontur	10
3.6 Menampilkan Seluruh Hasil	11
3.7 Hasil Pengujian	12
BAB IV	13
PENUTUP	13
DAFTAR PUSTAKA	14

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, maka rumusan masalah pada praktikum ini adalah sebagai berikut.

1. Bagaimana cara membaca dan menampilkan citra digital menggunakan Python dan OpenCV?
2. Bagaimana proses deteksi tepi menggunakan metode Canny pada sebuah citra digital?
3. Bagaimana proses pencarian kontur berdasarkan hasil deteksi tepi?
4. Bagaimana hasil yang diperoleh setelah dilakukan proses deteksi tepi dan deteksi kontur pada citra?

1.2 Tujuan Masalah

Adapun tujuan dari praktikum ini adalah sebagai berikut.

1. Memahami proses pengolahan citra digital menggunakan Python dan OpenCV.
2. Mengimplementasikan metode Canny Edge Detection untuk mendeteksi tepi pada citra.
3. Mengetahui proses pencarian kontur menggunakan fungsi yang tersedia pada OpenCV.
4. Menganalisis hasil deteksi tepi dan kontur pada citra digital.

1.3 Manfaat Masalah

Praktikum ini diharapkan dapat memberikan beberapa manfaat, baik secara teoritis maupun praktis. Secara teoritis, praktikum ini dapat menambah pemahaman mengenai konsep dasar pengolahan citra digital, khususnya pada proses deteksi tepi dan deteksi kontur. Selain itu, praktikum ini juga memberikan pemahaman mengenai penggunaan library OpenCV sebagai salah satu tools yang banyak digunakan dalam bidang computer vision.

Secara praktis, hasil praktikum dapat menjadi dasar dalam pengembangan aplikasi pengolahan citra yang lebih kompleks, seperti sistem pengenalan objek, pengenalan wajah, pengolahan citra medis, maupun sistem keamanan berbasis kamera. Selain itu, praktikum ini juga melatih kemampuan dalam mengimplementasikan teori ke dalam bentuk program menggunakan bahasa pemrograman Python.

BAB II

LANDASAN TEORI

2.1 Pengolahan Citra Digital

Pengolahan citra digital merupakan salah satu bidang dalam ilmu komputer yang mempelajari cara mengolah sebuah gambar menggunakan komputer agar informasi yang terdapat di dalam gambar tersebut dapat diperoleh atau ditampilkan dengan lebih baik. Saat ini pengolahan citra sudah banyak digunakan di berbagai bidang, seperti dunia kesehatan, industri, keamanan, hingga teknologi kecerdasan buatan. Dengan adanya pengolahan citra, sebuah gambar dapat diperbaiki kualitasnya, dianalisis, bahkan digunakan sebagai dasar dalam pengambilan keputusan oleh sistem komputer.

Sebuah citra digital tersusun atas kumpulan piksel (pixel) yang membentuk sebuah gambar. Setiap piksel memiliki nilai tertentu yang menunjukkan tingkat kecerahan atau warna. Semakin banyak jumlah piksel pada sebuah gambar, maka semakin tinggi pula kualitas gambar tersebut. Dalam proses pengolahan citra, komputer akan mengolah nilai pada setiap piksel sesuai dengan tujuan yang diinginkan, misalnya untuk memperjelas gambar, mendeteksi objek, atau mengubah posisi objek.

Pada praktikum UAS ini, pengolahan citra digunakan untuk melakukan transformasi geometri terhadap sebuah gambar menggunakan bahasa pemrograman Python dan library OpenCV. Proses tersebut dilakukan tanpa mengubah isi gambar, tetapi hanya mengubah posisi, ukuran, atau arah dari objek yang terdapat pada gambar.

2.2 Transformasi Geometri Citra

Transformasi geometri merupakan proses mengubah letak suatu objek pada citra tanpa mengubah bentuk maupun informasi yang dimiliki objek tersebut. Transformasi ini dilakukan dengan cara mengubah koordinat setiap piksel menggunakan suatu persamaan matematika. Hasil akhirnya dapat berupa gambar yang bergeser, diputar, diperbesar, diperkecil, maupun dicerminkan.

Transformasi geometri termasuk salah satu operasi dasar yang cukup sering digunakan dalam pengolahan citra. Teknik ini banyak diterapkan pada sistem pengenalan wajah, pengolahan citra satelit, OCR (Optical Character Recognition), hingga pelatihan model Artificial Intelligence. Selain itu, transformasi geometri juga sering dimanfaatkan sebagai proses awal (pre-processing) sebelum dilakukan analisis citra lebih lanjut.

Pada praktikum yang dilakukan, transformasi geometri diterapkan menggunakan beberapa metode, yaitu translasi, rotasi, scaling, dan flipping. Masing-masing metode menghasilkan perubahan yang berbeda sesuai dengan tujuan transformasi yang dilakukan.

2.3 Translasi (Translation)

Translasi merupakan proses menggeser posisi suatu objek pada citra ke arah tertentu tanpa mengubah ukuran maupun bentuk objek tersebut. Seluruh piksel pada gambar akan berpindah dengan jarak yang sama sehingga bentuk gambar tetap dipertahankan. Arah pergeseran dapat dilakukan ke kanan, kiri, atas, maupun bawah sesuai nilai koordinat yang diberikan.

Pada OpenCV, translasi dilakukan menggunakan fungsi **warpAffine()** dengan bantuan matriks transformasi berukuran 2×3 . Nilai translasi pada sumbu X menentukan seberapa jauh gambar bergeser ke arah horizontal, sedangkan nilai pada sumbu Y menentukan pergeseran ke arah vertikal.

Pada praktikum ini proses translasi digunakan untuk memindahkan posisi gambar sehingga dapat terlihat bagaimana perubahan koordinat memengaruhi hasil akhir citra.

2.4 Rotasi (Rotation)

Rotasi adalah proses memutar citra terhadap suatu titik pusat. Umumnya titik pusat yang digunakan adalah bagian tengah gambar agar objek tetap berada di dalam bidang citra. Besarnya putaran dinyatakan dalam satuan derajat. Jika nilai sudut bernilai positif maka gambar diputar berlawanan arah jarum jam, sedangkan nilai negatif akan menghasilkan putaran searah jarum jam.

Rotasi sering digunakan pada berbagai aplikasi pengolahan citra, misalnya untuk memperbaiki posisi dokumen hasil pemindaian, mengenali objek yang memiliki orientasi berbeda, maupun proses augmentasi data pada machine learning.

Dalam Python, proses rotasi dilakukan menggunakan fungsi **getRotationMatrix2D()**, kemudian hasil transformasi diterapkan dengan fungsi **warpAffine()** sehingga gambar dapat diputar sesuai sudut yang telah ditentukan.

2.5 Scaling

Scaling merupakan proses mengubah ukuran gambar menjadi lebih besar maupun lebih kecil. Proses ini dilakukan tanpa mengubah isi gambar, tetapi hanya mengubah ukuran setiap piksel berdasarkan faktor skala yang diberikan. Jika faktor skala lebih besar dari satu maka gambar akan diperbesar, sedangkan jika kurang dari satu maka gambar akan diperkecil.

Pada OpenCV, scaling dilakukan menggunakan fungsi **resize()**. Fungsi ini menyediakan beberapa metode interpolasi agar kualitas gambar tetap baik setelah proses pembesaran maupun pengecilan dilakukan.

Dalam praktikum, scaling digunakan untuk mengetahui bagaimana perubahan ukuran citra memengaruhi tampilan gambar serta kualitas hasil transformasi.

2.6 Flipping

Flipping merupakan proses mencerminkan suatu gambar terhadap sumbu tertentu sehingga menghasilkan tampilan seperti bayangan pada cermin. Transformasi ini terdiri dari tiga jenis, yaitu pencerminan horizontal, vertikal, dan gabungan keduanya.

Pada OpenCV, proses flipping dilakukan menggunakan fungsi **flip()**. Parameter yang diberikan menentukan arah pencerminan yang akan digunakan. Teknik ini banyak dimanfaatkan sebagai salah satu metode augmentasi data karena mampu menghasilkan variasi citra tanpa mengubah informasi utama dari objek tersebut.

Pada praktikum UAS, flipping digunakan untuk melihat perubahan orientasi gambar setelah dilakukan pencerminan pada masing-masing sumbu.

2.7 OpenCV dan Python

OpenCV merupakan library yang banyak digunakan dalam bidang pengolahan citra dan computer vision. Library ini menyediakan berbagai fungsi yang memudahkan proses membaca gambar, melakukan transformasi, mendeteksi objek, hingga menampilkan hasil pengolahan citra.

Bahasa pemrograman Python dipilih karena memiliki sintaks yang sederhana sehingga mudah dipahami oleh pemula. Selain itu, Python juga memiliki banyak library pendukung yang dapat digunakan dalam pengolahan citra, seperti NumPy, Matplotlib, dan OpenCV.

Pada praktikum ini Python digunakan sebagai bahasa pemrograman utama, sedangkan OpenCV digunakan untuk menjalankan seluruh proses transformasi geometri pada gambar. Dengan kombinasi keduanya, proses pengolahan citra dapat dilakukan dengan lebih mudah dan efisien.

BAB III

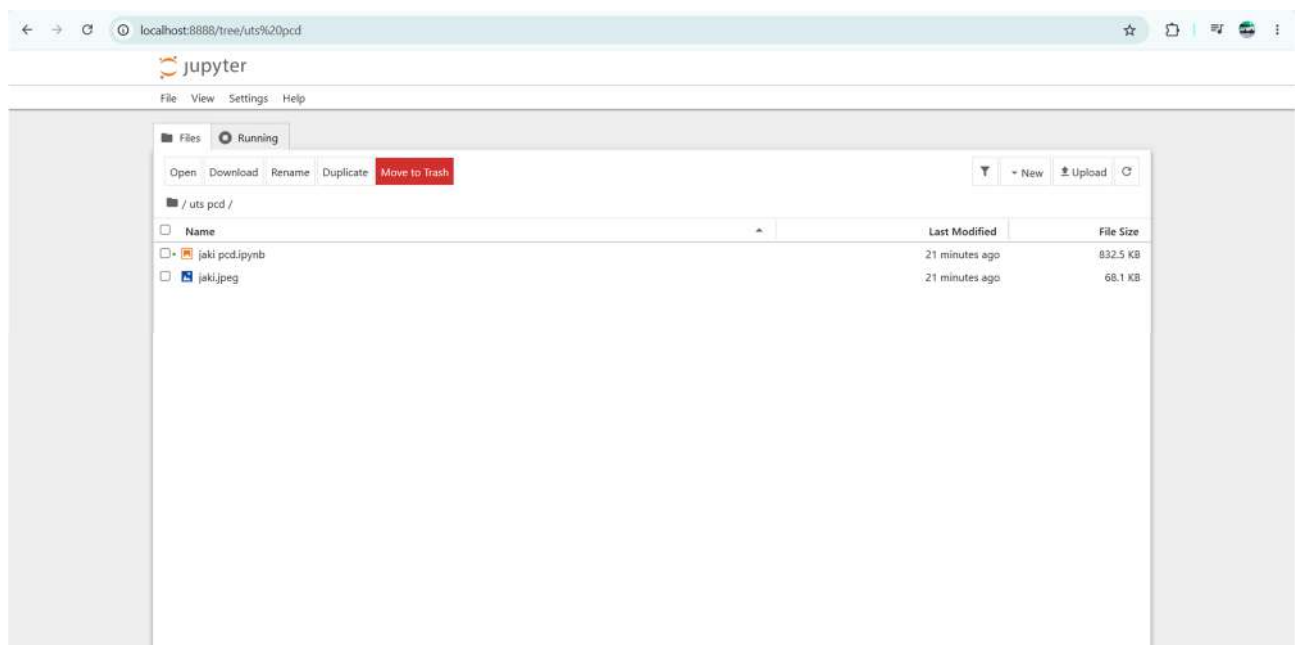
HASIL

3.1 Analisis Sistem

Pada praktikum ini dilakukan proses deteksi tepi (edge detection) menggunakan bahasa pemrograman Python dengan bantuan library OpenCV. Objek yang digunakan sebagai bahan pengujian adalah sebuah foto dengan nama **jaki.jpeg**. Tujuan dari proses ini yaitu untuk mengetahui bagaimana metode Canny dapat mendeteksi batas atau tepi suatu objek pada citra digital.

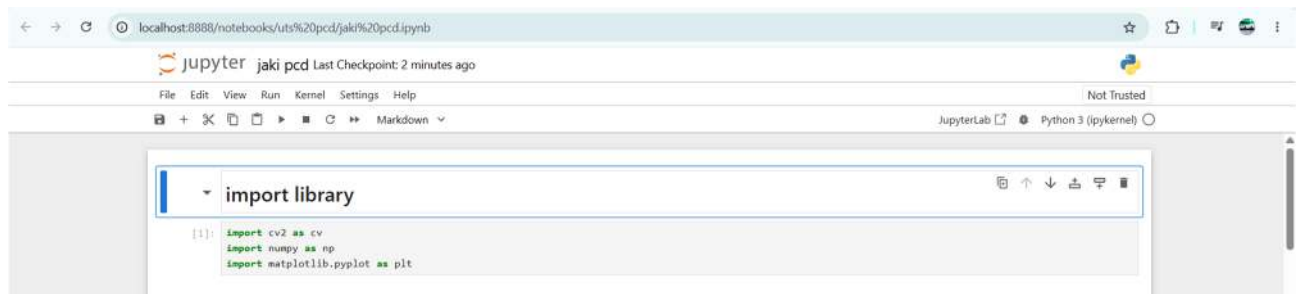
Secara umum proses yang dilakukan terdiri dari beberapa tahapan, dimulai dari mengimpor library yang dibutuhkan, membaca citra, mengubah format warna citra agar sesuai saat ditampilkan, melakukan konversi menjadi citra grayscale, menjalankan algoritma Canny Edge Detection, kemudian mencari kontur dari hasil deteksi tepi, dan terakhir menampilkan seluruh hasil pengolahan dalam satu tampilan sehingga lebih mudah dibandingkan.

Alur tersebut dibuat secara berurutan karena setiap proses saling berhubungan. Misalnya proses pencarian kontur tidak dapat dilakukan apabila sebelumnya belum dilakukan deteksi tepi menggunakan metode Canny. Oleh karena itu setiap tahapan memiliki fungsi masing-masing yang akan mempengaruhi hasil akhir.



3.2 Import Library

Tahap pertama adalah mengimpor beberapa library yang dibutuhkan dalam proses pengolahan citra.



Library **OpenCV (cv2)** digunakan sebagai library utama dalam pengolahan citra digital. Library ini menyediakan banyak fungsi seperti membaca gambar, mengubah warna gambar, mendeteksi tepi, hingga mencari kontur objek.

Library **NumPy** digunakan untuk mengolah data berupa matriks. Pada dasarnya gambar digital merupakan kumpulan piksel yang disusun dalam bentuk array dua dimensi maupun tiga dimensi sehingga NumPy sangat membantu proses pengolahan data tersebut.

Sedangkan **Matplotlib** digunakan untuk menampilkan gambar hasil proses agar dapat diamati secara visual. Dengan bantuan library ini hasil setiap tahapan dapat dibandingkan secara langsung.

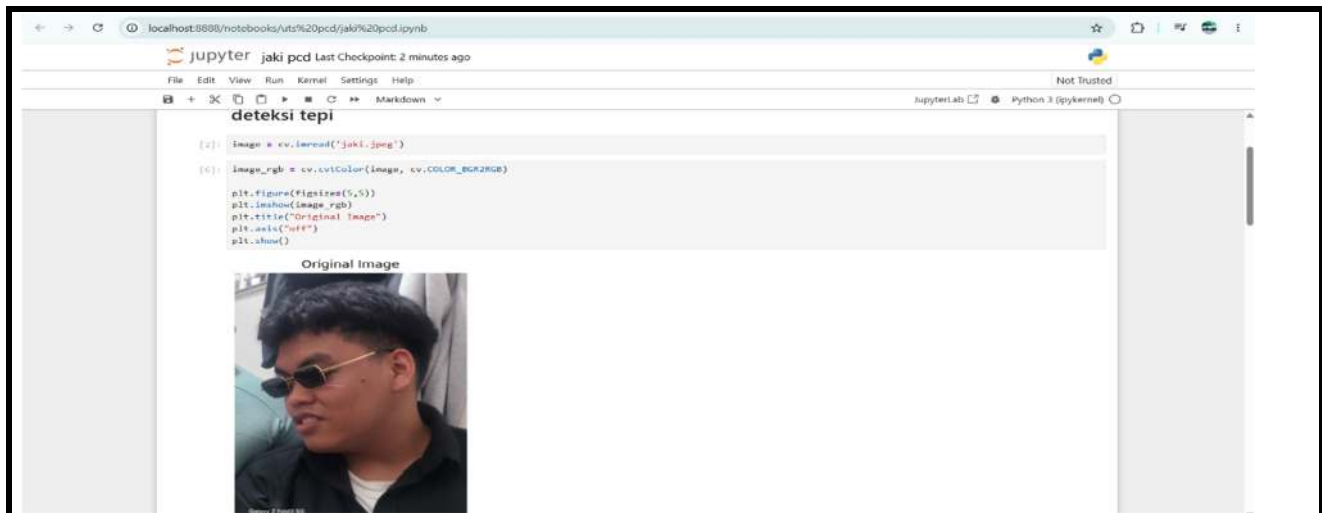
Tahap import library merupakan langkah awal yang wajib dilakukan sebelum menjalankan program. Tanpa library tersebut sebagian besar fungsi pengolahan citra tidak dapat digunakan.

3.3 Membaca Citra

Setelah seluruh library berhasil diimpor, langkah berikutnya yaitu membaca file gambar yang akan diproses.

Perintah di atas digunakan untuk mengambil file gambar dari folder kerja Jupyter Notebook. File yang digunakan bernama **jaki.jpeg**.

Kemudian dilakukan konversi warna agar gambar dapat ditampilkan dengan benar menggunakan Matplotlib.



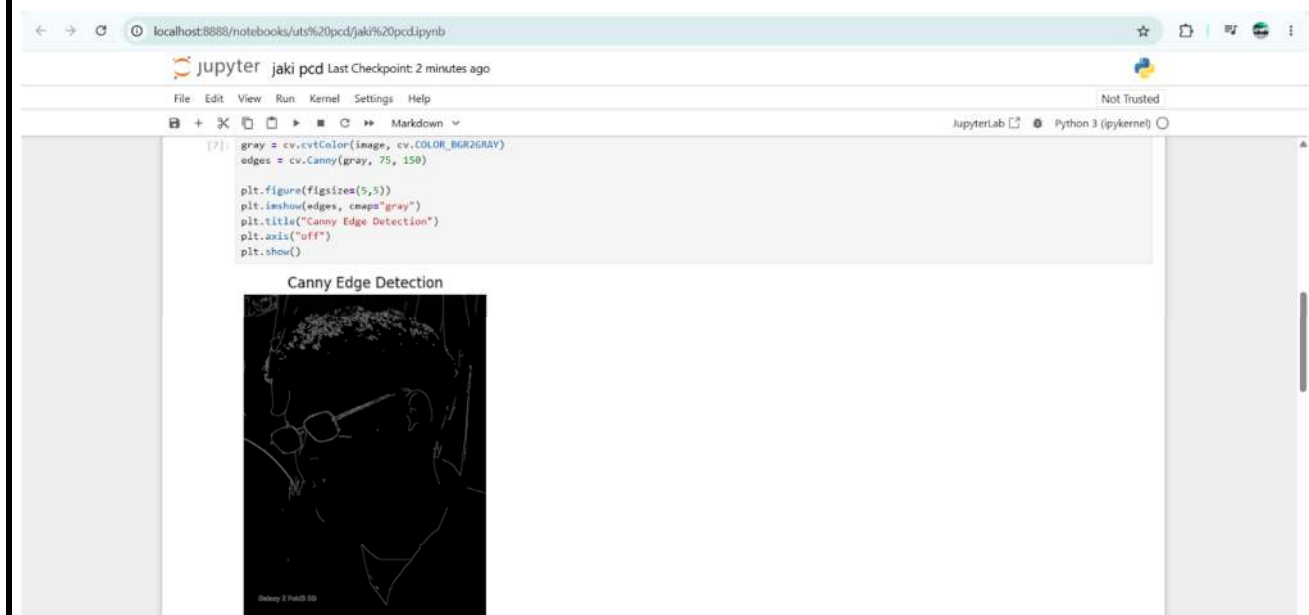
OpenCV secara default membaca gambar dalam format **BGR**, sedangkan Matplotlib menggunakan format **RGB**. Apabila gambar langsung ditampilkan tanpa dilakukan konversi warna maka hasil warna akan terlihat tidak sesuai dengan gambar asli.

Setelah dikonversi, gambar kemudian ditampilkan menggunakan fungsi **imshow()** sehingga dapat dipastikan bahwa gambar berhasil dibaca oleh program.

Berdasarkan hasil pengujian, citra asli berhasil ditampilkan dengan baik sehingga proses dapat dilanjutkan ke tahap berikutnya.

3.4 Proses Deteksi Tepi Menggunakan Metode Canny

Tahapan inti dari praktikum ini yaitu melakukan deteksi tepi menggunakan algoritma Canny.



Sebelum menjalankan algoritma Canny, gambar terlebih dahulu diubah menjadi citra grayscale. Hal ini dilakukan karena algoritma Canny bekerja berdasarkan perubahan intensitas cahaya sehingga tidak memerlukan informasi warna.

Selanjutnya fungsi **cv.Canny()** dijalankan dengan parameter **75** sebagai nilai threshold bawah dan **150** sebagai threshold atas.

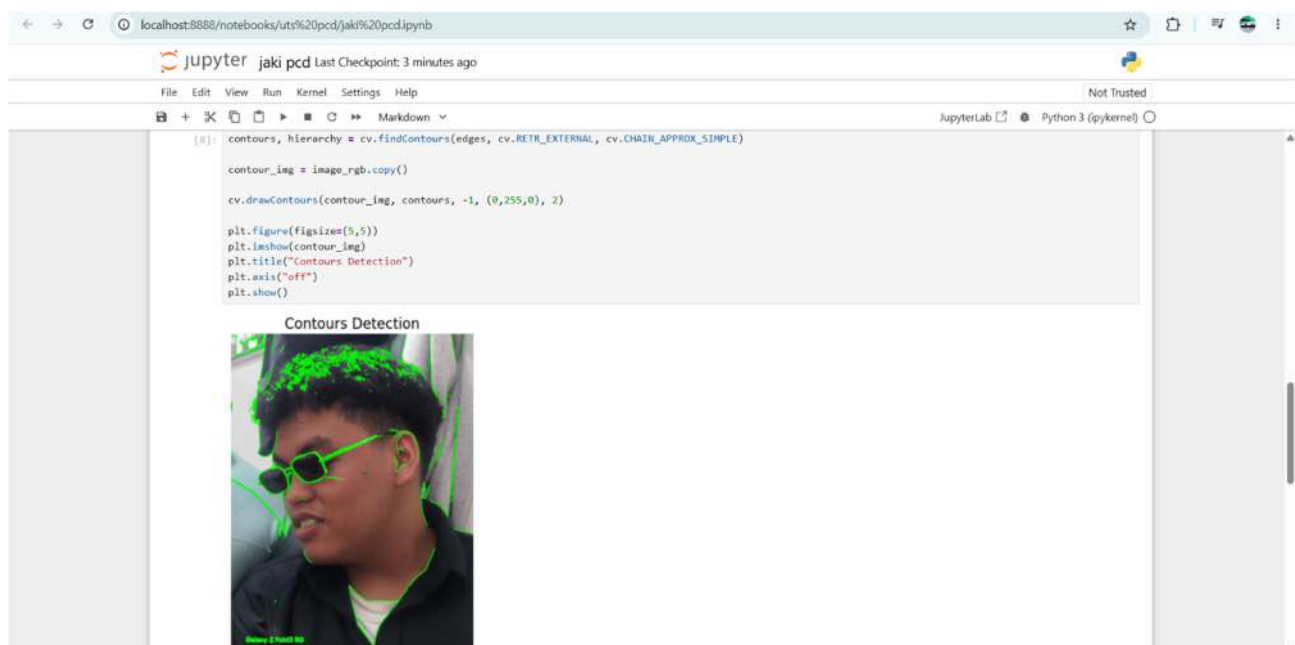
Nilai threshold tersebut menentukan sensitivitas proses pendeteksian tepi. Jika nilai threshold terlalu kecil maka banyak noise yang ikut terdeteksi sebagai tepi. Sebaliknya apabila threshold terlalu besar maka sebagian garis tepi dapat hilang.

Hasil pengujian menunjukkan bahwa garis-garis tepi pada objek wajah, rambut, kacamata, pakaian, hingga latar belakang berhasil dikenali dengan cukup baik. Garis yang memiliki perubahan intensitas tinggi akan terlihat berwarna putih sedangkan area lain tetap berwarna hitam.

Metode Canny dipilih karena mampu menghasilkan garis tepi yang lebih tipis dan jelas dibandingkan metode deteksi tepi sederhana seperti Sobel atau Prewitt.

3.5 Proses Pencarian Kontur

Setelah proses deteksi tepi selesai dilakukan, langkah berikutnya yaitu mencari kontur dari objek yang telah terdeteksi.



Fungsi **findContours()** digunakan untuk mencari seluruh garis luar objek berdasarkan hasil deteksi tepi sebelumnya.

Parameter **RETR_EXTERNAL** digunakan agar program hanya mengambil kontur bagian luar sehingga hasil menjadi lebih sederhana dan tidak terlalu banyak.

Sedangkan parameter **CHAIN_APPROX_SIMPLE** berfungsi mengurangi jumlah titik pada kontur sehingga penggunaan memori menjadi lebih efisien tanpa mengubah bentuk objek secara signifikan.

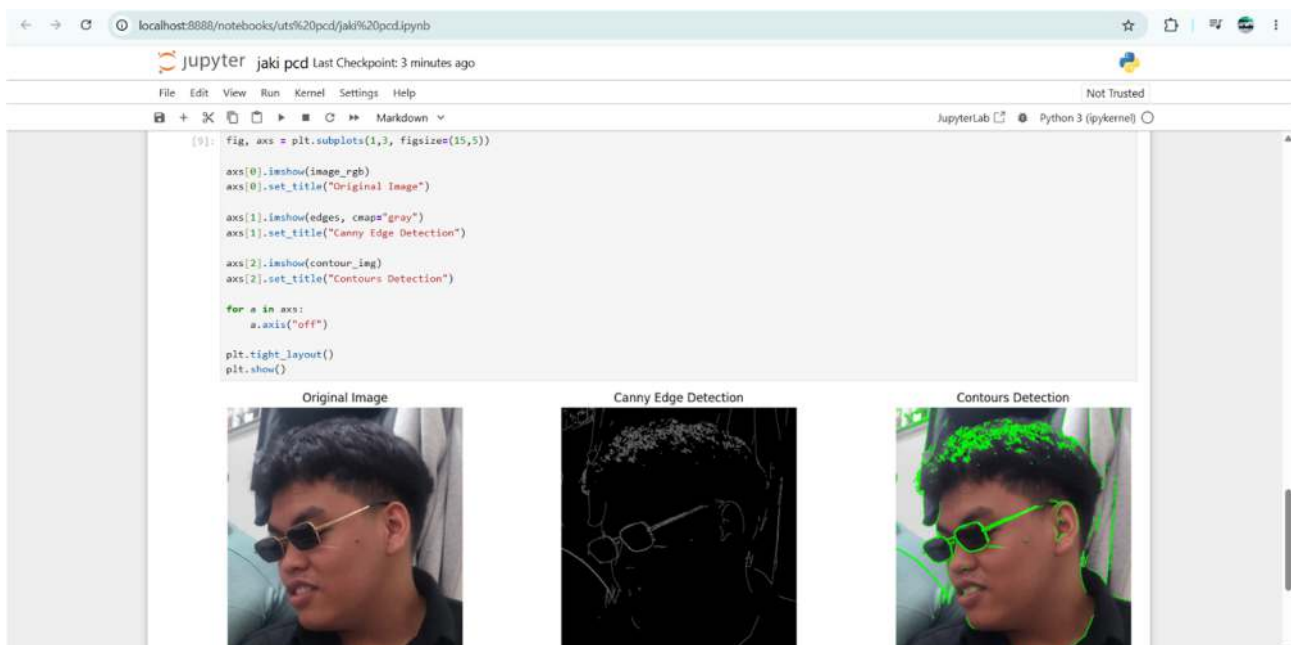
Setelah kontur berhasil diperoleh, fungsi **drawContours()** digunakan untuk menggambarkan seluruh garis kontur pada gambar asli menggunakan warna hijau.

Dari hasil yang diperoleh terlihat bahwa bagian rambut, wajah, kacamata, bahu, hingga beberapa bagian latar belakang berhasil diberikan garis kontur. Hal ini menunjukkan bahwa metode Canny mampu menyediakan informasi tepi yang cukup baik sebagai dasar pencarian kontur.

3.6 Menampilkan Seluruh Hasil

Tahap terakhir pada praktikum ini adalah menampilkan seluruh hasil pengolahan citra dalam satu tampilan. Tujuannya agar hasil dari setiap proses dapat dibandingkan secara langsung, mulai dari gambar asli, hasil deteksi tepi menggunakan metode Canny, hingga hasil pencarian kontur. Dengan cara ini, perubahan yang terjadi pada setiap tahapan akan lebih mudah diamati dan dianalisis.

Kode program yang digunakan adalah sebagai berikut.



Pada kode di atas, fungsi **plt.subplots()** digunakan untuk membuat tiga area tampilan dalam satu baris. Masing-masing area berisi gambar yang berbeda, yaitu citra asli, hasil deteksi tepi, dan hasil kontur. Susunan ini memudahkan pengguna dalam melihat perbedaan hasil pada setiap tahap pengolahan citra.

Gambar pertama menampilkan citra asli yang belum mengalami proses apa pun sehingga dapat dijadikan sebagai acuan. Gambar kedua menunjukkan hasil deteksi tepi menggunakan algoritma Canny, di mana garis-garis tepi objek terlihat berwarna putih dengan latar belakang hitam. Selanjutnya, gambar ketiga memperlihatkan hasil pencarian kontur yang digambar menggunakan garis berwarna hijau pada citra asli sehingga bentuk objek menjadi lebih jelas.

Selain itu, program juga menggunakan perulangan **for** untuk menghilangkan sumbu koordinat pada setiap gambar agar tampilan menjadi lebih rapi. Fungsi **plt.tight_layout()** digunakan untuk mengatur jarak antar gambar sehingga judul dan gambar tidak saling bertumpuk, sedangkan **plt.show()** berfungsi menampilkan seluruh hasil ke layar.

Berdasarkan hasil yang diperoleh, penyajian tiga gambar dalam satu tampilan sangat membantu dalam proses analisis. Pengguna dapat melihat dengan jelas bagaimana gambar asli diolah menjadi citra tepi, kemudian dilanjutkan menjadi hasil kontur. Dari pengujian tersebut terlihat bahwa metode Canny mampu menghasilkan deteksi tepi yang cukup baik sehingga proses pencarian kontur juga memberikan hasil yang jelas pada objek utama di dalam gambar.

3.7 Hasil Pengujian

Berdasarkan hasil pengujian yang telah dilakukan menggunakan gambar **jaki.jpeg**, seluruh tahapan program berhasil dijalankan tanpa mengalami error.

Hasil deteksi tepi memperlihatkan bahwa objek utama seperti rambut, wajah, kacamata, pakaian, dan sebagian latar belakang berhasil dikenali oleh algoritma Canny. Garis tepi yang dihasilkan terlihat cukup tipis sehingga memudahkan proses pencarian kontur.

Selanjutnya proses pencarian kontur juga berhasil menampilkan garis hijau yang mengikuti bentuk objek. Walaupun masih terdapat beberapa garis pada bagian latar belakang, secara keseluruhan objek utama tetap dapat dikenali dengan baik.

Hal tersebut menunjukkan bahwa metode Canny memiliki kemampuan yang cukup baik dalam mendeteksi perubahan intensitas cahaya pada citra digital. Namun kualitas hasil tetap dipengaruhi oleh pencahayaan gambar, resolusi citra, dan nilai threshold yang digunakan saat proses deteksi.

BAB IV

PENUTUP

Berdasarkan hasil praktikum yang telah dilakukan, seluruh tahapan pengolahan citra menggunakan metode Canny Edge Detection berhasil dijalankan dengan baik. Proses dimulai dari membaca citra, mengubah format warna, mengonversi citra menjadi grayscale, melakukan deteksi tepi, mencari kontur, hingga menampilkan seluruh hasil pengolahan dalam satu tampilan. Setiap tahapan memiliki peran penting dalam menghasilkan proses deteksi tepi yang sesuai dengan tujuan praktikum.

Hasil pengujian menggunakan gambar **jaki.jpeg** menunjukkan bahwa metode Canny mampu mendeteksi garis tepi pada objek utama seperti rambut, wajah, kacamata, dan pakaian dengan cukup jelas. Selain itu, proses pencarian kontur juga berhasil menggambarkan batas objek pada citra sehingga bentuk objek menjadi lebih mudah dikenali. Meskipun masih terdapat beberapa garis pada bagian latar belakang akibat tekstur dan pencahayaan, secara keseluruhan hasil yang diperoleh sudah sesuai dengan tujuan pengolahan citra.

Dari pembahasan yang telah dilakukan dapat disimpulkan bahwa penggunaan library OpenCV sangat membantu dalam proses implementasi pengolahan citra digital karena menyediakan fungsi-fungsi yang lengkap dan mudah digunakan. Hasil praktikum ini juga menunjukkan bahwa metode Canny Edge Detection merupakan salah satu metode yang efektif untuk mendeteksi tepi objek dan dapat menjadi dasar dalam berbagai aplikasi pengolahan citra maupun computer vision.

DAFTAR PUSTAKA

- *Gangal, A., Kumar, P., & Kumari, S. (2021). Complete Scanning Application Using OpenCV. arXiv.*
- *Su, Z., Liu, W., Yu, Z., Hu, D., Liao, Q., Tian, Q., Pietikäinen, M., & Li, L. (2021). Pixel Difference Networks for Efficient Edge Detection. arXiv.*
- *OpenCV. (2024). Canny Edge Detection.*
- *OpenCV. (2024). Image Processing in OpenCV.*
- *Python Software Foundation. (2024). Python 3 Documentation.*